

Aula 1 — Fundamentos de Testes Mobile

"3M apps na loja, maioria abandonado após 1º uso. Por que QA mobile falha tanto?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 21/05/2026

Bem-vindos!

"Bem-vindo à jornada de excelência em testes de aplicações mobile. Este curso é sua porta de entrada para dominar as técnicas, ferramentas e estratégias que garantem qualidade superior dos aplicativos móveis modernos."

— frase de abertura herdada das ofertas anteriores. Esta é a **4ª oferta**.

App de referência oficial — TestesQAMobile

Vocês vão testar **um app real, em produção, em App Store e Play Store** — feito por mim pra esta disciplina.

- 🍏 App Store: apps.apple.com/br/app/testes-qa-mobile/id6755933674
- 🤖 Play Store: play.google.com/store/apps/details?id=com.apptestesmobile
- 📦 Bundle: `com.apptestesmobile`

Instale via lojas (gratuito). Código-fonte é privado — vocês testam o app de fora, como QA real.

35 exercícios em 12 categorias com bugs propositalmente colocados:

1. Funcional · 2. Usabilidade · 3. UI · 4. Compatibilidade · 5. Performance · 6. Conectividade · 7. Instalação · 8. Interrupção · 9. Localização · 10. Gestos · 11. Acessibilidade · 12. Segurança

"Quando concluírem a disciplina, dizem que testaram um app que está em produção. Coloquem no LinkedIn."

Agenda da aula (3h30)

1. **Bloco 1 (90min)** — Ecosystema mobile + Pirâmide de Knott
2. **Intervalo (15min)**
3. **Bloco 2 (90min)** — Hands-on: análise de cobertura em app open source
4. **Wrap-up (15min)** — Atividade 1 + leituras

Objetivos de aprendizagem

- **Compreender** o ecossistema mobile em 2026 (fragmentação real, sensores, conectividade)
- **Aplicar** pirâmide de testes mobile (Knott) com critério
- **Diferenciar** testes manuais, exploratórios, unitários, integração, UI E2E
- **Avaliar** quando usar real device vs emulador vs cloud farm
- **Estruturar** programa de beta testing com TestFlight + Firebase

Por que QA mobile é diferente

Aspecto	Web	Mobile
Plataformas	Chromium, Firefox, WebKit	iOS + Android (+ KaiOS, HarmonyOS)
Devices	Variação tela/CPU	+ sensores, GPS, biometria, câmera, NFC
Conectividade	WiFi/Eth	+ 4G/5G, modo avião, perda intermitente
Atualização	Refresh	Build + store + user opt-in
Permissões	Cookies, storage	+ camera, location, contacts, notifications
Background	Limitado	App vida própria com lifecycle complexo

Mobile tem ordem de magnitude mais variáveis. QA precisa adaptar.

Crise da qualidade mobile

3 milhões+ de aplicativos disponíveis nas lojas.

77% dos apps são abandonados após o **primeiro uso**.

Razões:

- UX confusa
- Crashes
- Lentidão
- Permissões abusivas
- Bugs visíveis em primeiro contato

Investimento em QA é diferencial competitivo real, não custo.

Fragmentação Android

- **10.000+ devices** Android ativos no mercado
- **8 versões** suportadas comercialmente (5.0–14)
- **5 fabricantes** principais (Samsung, Xiaomi, Oppo, Vivo, Motorola)
- **Skins customizadas** (One UI, MIUI, ColorOS) que mudam comportamento
- **Resoluções:** 320p até 4K, aspect ratios variados, dobráveis

Resultado: bugs específicos de marca/modelo são realidade.

Fragmentação iOS

- **Menos devices**, mas **30+ modelos** ativos (iPhone 7+ ainda em uso)
- **Versões iOS**: tipicamente 3 grandes coexistem (iOS 16, 17, 18 em 2026)
- **Aspect ratios**: dynamic island, notch, home button (legacy)
- **Capacidade de chip**: A11 (iPhone X) até A18 Pro varia em 4x

Aparenta ser mais simples que Android, mas **regressões iOS-específicas** existem.

Pirâmide de testes — Cohn (clássica web)

E2E	10%
Service	20%
Unit	70%

Funciona bem em web. **Problema em mobile:** testes E2E mobile são mais frágeis e mais valiosos simultaneamente.

Pirâmide de Knott (mobile)

Daniel Knott, em *Hands-On Mobile App Testing* (Pearson 2015):

E2E (UI)	10%
Integration	20%
Unit	30%
Manual / Expl.	40% ← maior peso!

Peso maior em exploratório manual. Por quê? Variáveis mobile (sensores, gestos, conectividade) escapam de automação tradicional.

Testing Trophy — Kent C. Dodds

Alternativa moderna proposta para JavaScript:

E2E
Integration
Unit
Static

← maior peso!

Aplicável a apps web e mobile com data fetching estruturado. Cada estratégia tem contexto.

Tipos de teste mobile

1. **Funcional** — feature funciona conforme spec
2. **Usabilidade** — UX coerente, intuitiva, acessível
3. **Integração** — módulos conversam corretamente
4. **Desempenho** — cold start, FPS, memória, bateria
5. **Segurança** — OWASP Mobile Top 10, MASVS
6. **Estabilidade** — crash-free, longevidade
7. **A/B** — variantes, experimentos
8. **Compatibilidade** — devices, OS versions
9. **Acessibilidade** — VoiceOver, TalkBack
10. **Localização** — i18n, RTL, formatos

Real device vs emulator/simulator vs cloud

Critério	Real device	Emulator/Sim	Cloud farm
Setup	Caro, cabeado	Grátis, local	Pago / free tier
Realismo	100%	~80%	100%
Sensores reais	Sim	Mockáveis	Limitados
Velocidade	Rápido	Mais rápido	Mais lento (rede)
Cobertura devices	Limitada	1 modelo	100s de modelos
Custo escala	Alto	Zero	Por minuto

Estratégia ótima: mix dos três conforme cenário.

Cloud farms — opções 2026

- **Firebase Test Lab** — 100min/mês free, integração Google nativa
- **BrowserStack App Live + App Automate** — cobertura ampla, premium
- **LambdaTest** — alternativa BrowserStack, mais barata
- **Sauce Labs** — enterprise, integração CI rica
- **Kobiton** — focado em devices reais
- **AWS Device Farm** — para quem está no ecossistema AWS

Estratégia consagrada: **top 10–15 devices por share** (StatCounter) cobrem ~80% da base.

Beta testing — frameworks

iOS: TestFlight (Apple)

- Até 10k beta testers
- Build expira em 90 dias
- Feedback integrado

Android: Firebase App Distribution + Google Play Internal Track

- Internal/Closed/Open track
- Distribuição via email/link
- Feedback via Firebase

Beta com usuários reais captura bugs que QA interno não vê.

Métricas de qualidade mobile

- **Crash-free users:** > 99.5% target
- **ANR rate** (Android Not Responding): < 0.1%
- **App size:** ~50–80MB ideal (cresce com features mas não infinitamente)
- **Cold start:** < 2s
- **Frame rate:** 60fps sustentado, 120fps em flagships
- **Bateria:** -X% por hora (variável por app)

Dashboards essenciais: Firebase Performance, Sentry, Crashlytics.

Hands-on — análise de cobertura (90min)

Roteiro

1. Escolher app open source mobile (5min)
 - WordPress Mobile (RN), Wikipedia mobile (nativo), OWASP iGoat
2. Clonar repo (5min)
3. Mapear estratégia atual (15min)
 - Que tipos de teste existem?
 - Que ferramentas usam?
 - Tem CI configurado?
4. Identificar gaps (15min)
5. Propor melhorias com priorização justificada (20min)
6. Discussão em duplas (15min)
7. Plenária (15min)

Pitfalls comuns

- ✗ **Pirâmide Cohn aplicada cegamente em mobile** — falta peso em exploratório
- ✗ **Só testar em emulador** — esconde problemas reais (touch latency, sensor noise)
- ✗ **Beta testing improvisado** — sem charter, sem feedback estruturado
- ✗ **Métricas inexistentes** — sem dashboard, debate vira opinião
- ✗ **Considerar testing manual "antiquado"** — em mobile, ainda é peça central

Atividade 1: Análise de Cobertura (15 pts)

Entrega: até 27/05.

Tarefa:

1. Escolher app open source ativo (≥ 100 stars GitHub)
2. **Mapear estratégia atual** (testes existentes, ferramentas, CI)
3. **Análise de gaps** — o que falta, por que importa
4. **Proposta de melhorias** com priorização
5. Mínimo **3 referências acadêmicas**

Critérios detalhados de avaliação: Canvas (módulo Atividade 1).

Leituras obrigatórias (pré-Aula 2)

- **Knott, D.** (2014) *The Mobile Test Pyramid*. Ministry of Testing.
- **Knott, D.** (2018) *3 Steps to Creating a Mobile Testing Strategy*. Applause.
- **Linares-Vásquez et al.** (2017) *Continuous, Evolutionary and Large-Scale: A New Perspective for Automated Mobile App Testing*. ICSME (preprint).
- **Fowler, M.** (2018) *The Practical Test Pyramid*.
- **Google Testing Blog** — *Just Say No to More End-to-End Tests* (2015) + *Test Sizes* (2010).
- **ISTQB** — Foundation Level Syllabus v4.0 (overview).

PDFs em [material-de-apoio/aula-01/](#) (todas fontes públicas).

Próxima aula (28/05)

Aula 2 — Setup, Manual Estruturado e Unit Testing Mobile

Pré-requisito:

- Atividade 1 entregue
- Xcode 16+ instalado
- Android Studio + AVD configurado
- ADB tools no PATH

Obrigado!

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith

Próxima quinta, 28/05, 19:00